

Arquitectura de ordenadores 2024/2025

Práctica 4

Jorge Jiménez Oropesa, Antonio Moroño Moreno

Grupo 1322, pareja 15

Ejercicio 0

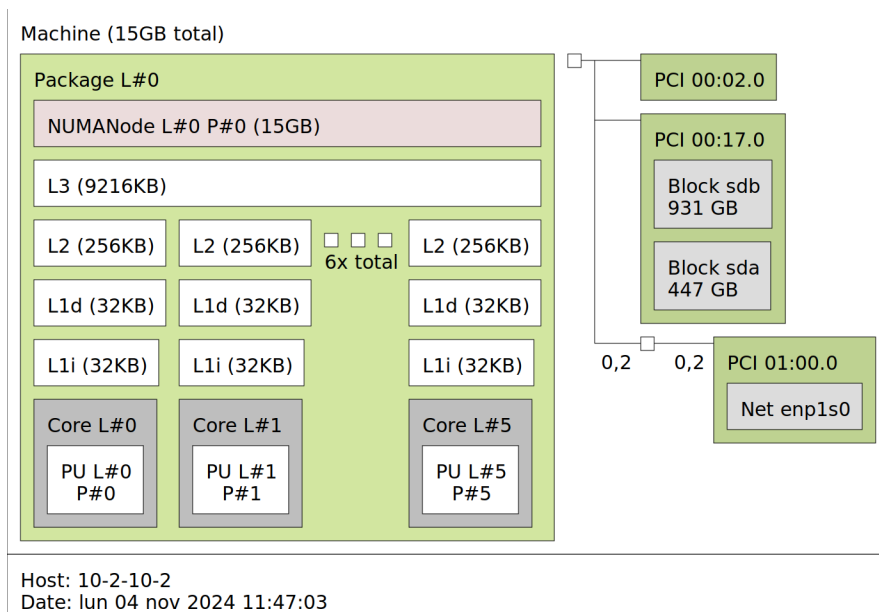
Apartado 1

Empleando el comando `cat /proc/cpuinfo` obtenemos la siguiente información sobre la caché:

```
siblings : 6
cpu cores : 6
```

Por lo tanto, observamos que el ordenador de los laboratorios cuenta con una CPU de 6 núcleos, y un total de 6 hilos físicos. De esto concluimos que cada núcleo dispone de un hilo. Por lo tanto, no hay hyperthreading.

Empleando la instrucción `lstopo`, por otro lado, obtenemos el siguiente diagrama:



Podemos ver que hay 6 cores, cada uno con una memoria caché L1i de 32 KB, otra caché L1d de 32 KB, y otra caché L2 de 256 KB. Además, hay una caché común a todos los cores de tipo L3 con 9216 KB. Además, podemos ver que la separación de datos e instrucciones está presente únicamente en el nivel L1, ya que se separan las cachés entre L1i (instructions) y L1d (data).

Ejercicio 1

1.1 ¿Se pueden lanzar más threads que cores tenga el sistema? ¿Tiene sentido hacerlo?

Sí se pueden lanzar más threads que cores tiene el sistema, pero no tiene sentido hacerlo.

1.2 ¿Cuántos threads debería utilizar en los ordenadores del laboratorio? ¿Y en su propio equipo?

En general, lo óptimo es lanzar tantos threads como tengan en total los cores que vayamos a usar. En los ordenadores del laboratorio, como cada core tiene un thread físico, si usamos los 6 cores, lo mejor será lanzar 6 hilos para aprovechar los 6 hilos físicos de los que disponemos.

1.3 Modifique el programa omp1.c para utilizar las tres formas de elegir el número de threads y deduzca la prioridad entre ellas.

Tras hacer varias modificaciones al archivo omp1.c hemos podido comprobar que la prioridad de cada forma de declarar el número de threads es, de mayor prioridad a menor:

- Cláusula en la declaración paralela (`#pragma omp parallel num_threads(numthr)`)
- Usando la función `omp_set_num_threads(int num_threads)`
- Con la variable de entorno `OMP_NUM_THREADS`

Para las siguientes preguntas, hemos compilado y ejecutado omp2. Esta es la salida obtenida, en la que nos basaremos para responder:

```
$ ./omp2
Inicio: a = 1,  b = 2,  c = 3
      &a = 0x7ffc96716db4, x      &b = 0x7ffc96716db8, &c
0x7ffc96716dbc

[Hilo 0]-1: a = -340918896,  b = 2,  c = 3
[Hilo 0]   &a = 0x7ffc96716d50, &b = 0x7ffc96716db8, &c
0x7ffc96716d4c
[Hilo 0]-2: a = 264630543,  b = 4,  c = 264630531
[Hilo 1]-1: a = 0,  b = 2,  c = 3
[Hilo 1]   &a = 0x73020dffd80, &b = 0x7ffc96716db8, &c
0x73020dffd7c
[Hilo 1]-2: a = 21,  b = 6,  c = 3
[Hilo 2]-1: a = 0,  b = 2,  c = 3
[Hilo 2]   &a = 0x73020d5ffd80, &b = 0x7ffc96716db8, &c
0x73020d5ffd7c
```

```
[Hilo 2]-2: a = 27,    b = 8,    c = 3
[Hilo 3]-1: a = 0,    b = 2,    c = 3
[Hilo 3]    &a = 0x73020cbffd80, &b = 0x7ffc96716db8, &c
0x73020cbffd7c
[Hilo 3]-2: a = 33,    b = 10,    c = 3

Fin: a = 1,          b = 10,    c = 3
      &a = 0x7ffc96716db4, &b = 0x7ffc96716db8, &c = 0x7ffc96716dbc
```

(Respuesta conjunta de 1.4, 1.5, 1.6)

1.4 ¿Cómo se comporta OpenMP cuando declaramos una variable privada?

1.5 ¿Qué ocurre con el valor de una variable privada al comenzar a ejecutarse la región paralela?

1.6 ¿Qué ocurre con el valor de una variable privada al finalizar la región paralela?

Atendiendo a la variable **a**, que se declara como privada, podemos observar que en todos los hilos cambia tanto el valor como la dirección de **a**. Esto sugiere que para cada hilo se reserva otro espacio en memoria para la variable, y no se inicializa con el valor original, ya que los valores de **a** en los hilos no coinciden con su valor previo.

Por otro lado, fijándonos en los valores mostrados por consola tras la paralelización, comprobamos que **a** no tiene el valor con el que terminó en ninguno de los hilos, si no que mantiene el valor '1' con el que se inicializó originalmente. Esto sugiere que cuando declaramos una variable privada, las instancias que se emplean en el bucle están completamente desvinculadas de la original, compartiendo sólo el nombre, y no afectándola de ninguna forma.

1.7 ¿Ocurre lo mismo con las variables públicas?

No. Atendiendo a **b** (shared) y **c** (first_private) vemos que se inicializan con los valores originales para cada hilo. Además, observamos que **b** comparte dirección de memoria entre todos los hilos. Por otro lado, comprobamos que al finalizar la ejecución paralela, los valores que le quedan a **b** y **c** son los asignados en el último hilo, en este caso el hilo 3.

Ejercicio 2

2.1 Ejecute la versión serie y entienda cuál debe ser el resultado para diferentes tamaños de vector.

Teóricamente, dados dos vectores de tamaño M , se realizan M sumas y M multiplicaciones, y como estas dos son las operaciones que principalmente determinan el tiempo de ejecución, es razonable asumir que la complejidad del algoritmo irá determinada por las mismas. Puesto que ambas son lineales, el algoritmo también lo será.

Tras ejecutar, comprobamos que efectivamente, al multiplicar por 10 el tamaño del vector, el tiempo de ejecución se multiplica aproximadamente por 10, tal y como conjeturamos.

```
Resultado: 100000.000000
```

```
Tiempo: 0.000237
```

```
Resultado: 1000000.000000
```

```
Tiempo: 0.003619
```

```
Resultado: 10000000.000000
```

```
Tiempo: 0.029955
```

2.2 Ejecute el código paralelizado con el pragma openmp y conteste en la memoria a las siguientes preguntas: ¿Es correcto el resultado? ¿Qué puede estar pasando?

Al ejecutar la versión base del código, vemos que devuelve un resultado incorrecto, y que además cambia entre ejecuciones. Esto se debe a que los múltiples hilos que se crean intentan acceder simultáneamente y de forma desincronizada a la variable `sum`, lo que genera condiciones de carrera y altera el correcto funcionamiento del algoritmo.

2.3 Modifique el código anterior y denomine el programa `pescalar_par2`. Esta versión deberá dar el resultado correcto utilizando donde corresponda alguno de los siguientes pragmas. ¿Puede resolverse con ambas directivas? Indique las modificaciones realizadas en cada caso. ¿Cuál es la opción elegida y por qué?

Como hemos discutido previamente, el problema original surge de escribir sin control sobre la variable `sum`. Al utilizar el pragma `critical` dentro del bucle `for`, logramos sincronizar los hilos, pues esta directiva fuerza al programa a que sólo un hilo pueda ejecutar simultáneamente esa sección de código. `Atomic`, por su parte, también funciona correctamente, ya que fuerza al programa a tratar las variables de forma sincronizada para evitar condiciones de carrera, tanto en la lectura como en la escritura.

Nos hemos decantado por esta última implementación porque consideramos que resuelve el problema de forma más específica que critical, ya que se limita a restringir el acceso a las variables, en vez de a todo el código del bucle for. Es cierto que en este caso es una única línea de código, pero creemos que es una buena práctica.

2.4 Modifique el código anterior y denomine el programa resultante pescalar_par3. Esta versión deber dar el resultado correcto utilizando donde corresponda alguno de los siguientes pragmas

La cláusula reduction(+:sum) permite que cada hilo mantenga una copia privada de la variable sum y realice la suma de manera independiente. Al final de la región paralela, OpenMP combina todas las sumas parciales en una sola operación de reducción.

#pragma omp atomic asegura que la operación de actualización de sum sea atómica, lo que significa que solo un hilo puede actualizar sum a la vez. Esto puede causar una contención significativa y reducir el paralelismo, ya que los hilos tendrán que esperar para actualizar sum.

Por lo tanto, creemos que la mejor opción es usar la cláusula reduction.

2.6 - 2.7 Análisis de tiempos de ejecución

Para este apartado, hemos desarrollado programa, pescalar_threshold_automated que calcula un threshold que cumpla las condiciones solicitadas de forma confiable.

El método principal sobre el que el programa opera es `test_block`. Este método recibe un tamaño de vector, y un número de iteraciones (`BLOCK_SIZE`). Mide el tiempo que el producto escalar monohilo tarda en ejecutar el número de iteraciones especificado, y después hace lo mismo para la versión multihilo.

El programa parte de un tamaño de vector inicial definido mediante un macro. Calcula los tamaños pequeño (`ceil(0.8*size)`) y grande (`ceil(1.2*size)`).

Empieza probando el tamaño pequeño. Para esto, ejecuta `test_block` `N_OF_BLOCKS` veces. Para considerar que el threshold es válido, será necesario que en todos los `test_block` la versión monohilo sea más rápida. De no ser así, se disminuirá el tamaño del vector y se volverá a empezar. Además, para agilizar la ejecución, la magnitud de esta disminución dependerá del número de bloques válidos. Así, si muchos bloques son válidos, se disminuirá poco, porque se asume que estamos cerca de un threshold válido. Si, en cambio, hay pocos bloques válidos, el salto será mayor.

Si todos los bloques son válidos (la versión monohilo es más rápida), entonces pasa a probar el tamaño grande, para la que procedemos de forma análoga, sólo que ahora consideraremos que un bloque es válido si la versión multihilo es más rápido, y si no todos los bloques son válidos aumentaremos el tamaño.

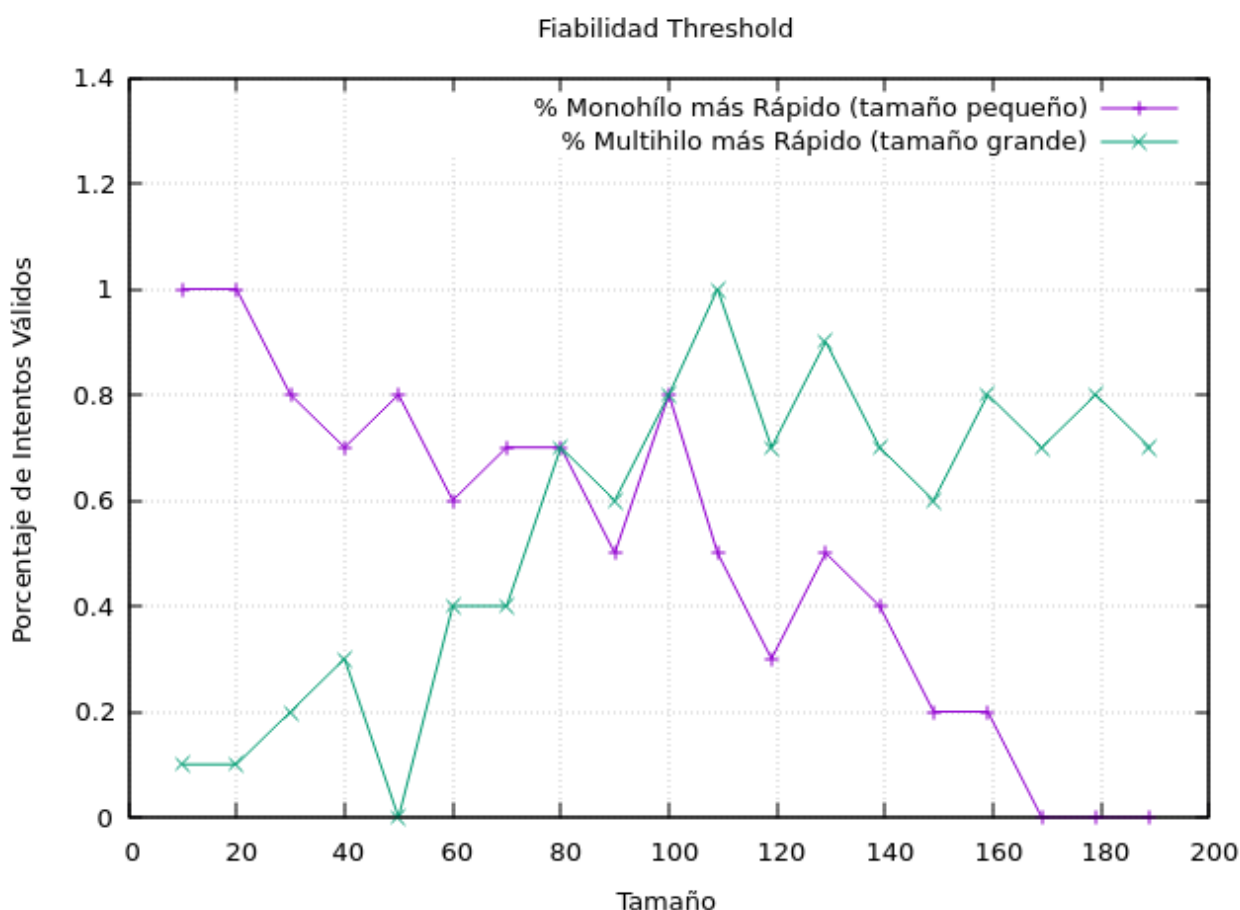
Hemos decidido emplear bloques porque así para tamaños muy pequeños de vector, que se ejecutan muy rápido, podemos tener más precisión y combatir las irregularidades que se puedan dar durante la ejecución. No obstante, el tamaño de bloque por defecto es 1, para ceñirnos a lo solicitado por la práctica.

De esta forma, el programa encuentra thresholds válidos de forma eficiente, y asegurándose de que una vez encuentra uno, este es confiable, pues resulta válido para múltiples iteraciones (`N_OF_BLOCKS` = 10, en nuestro caso).

Además, hemos creado un script de bash, `threshold.sh`, que ejecuta el programa y crea un gráfico con `gnuplot`. En el eje X distintos tamaños de vector, y en el eje Y el número de bloques válidos. Hay dos gráficas, una que representa. La violeta muestra, al multiplicar el tamaño del vector por 0.8 y ejecutar las versiones monohilo y multihilo, qué porcentaje de bloques son más rápidos para el monohilo (condición solicitada para el threshold). La gráfica verde muestra, al multiplicar el tamaño del vector por 1.2 y ejecutar las versiones monohilo y multihilo, qué porcentaje de bloques son más rápidos para el multihilo (condición solicitada para el threshold).

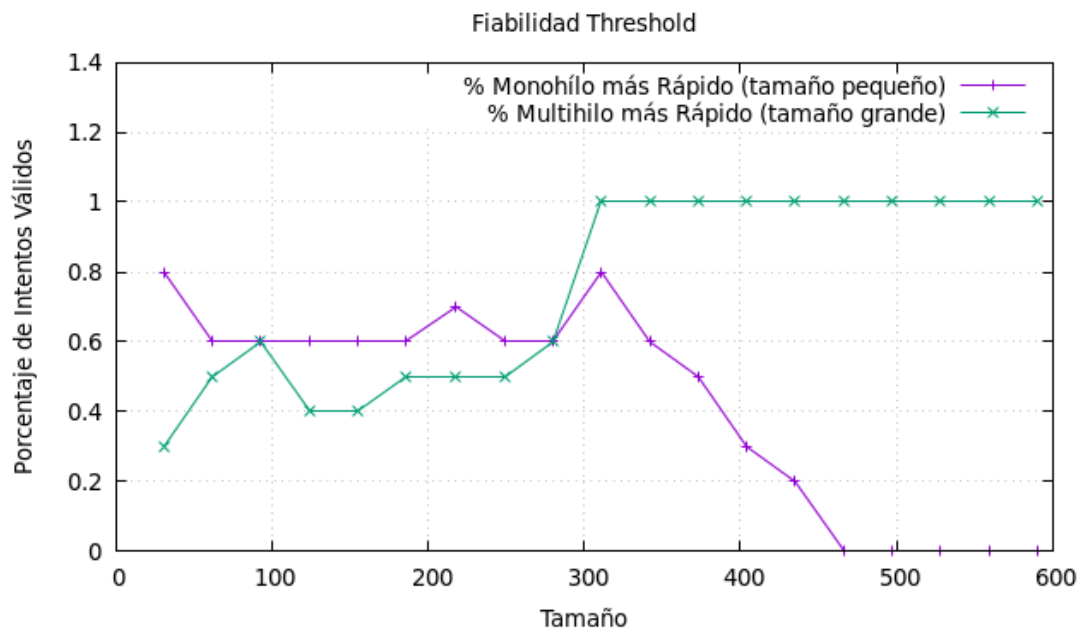
Es importante notar que primero se localiza el threshold tal que todos los bloques se validan, y luego vuelve a recalcularse para elaborar la gráfica. Al recalcularse, es posible (y, de hecho, normal) que no todos los bloques se validen por pura probabilidad, pero sí que se acercan al 100%, por lo que siguen siendo confiables.

Tras ejecutarlo, encuentra un threshold válido (100% de bloques válidos) en el tamaño 99. Esta es la gráfica que obtenemos al ejecutarlo para distintos tamaños de vector:



Como era de esperar, si el tamaño de vector es inferior al threshold, el monohilo es casi siempre más rápido que el multihilo para la versión x0.8, y tiende a ser más rápida que la multihilo en la versión x1.2 a medida que disminuimos el tamaño. Lo contrario ocurre si superamos el threshold: el monohilo tiende a ser más lento que el multihilo en la versión x0.8, y es casi siempre más lento en la versión x1.2.

Hemos replicado esta prueba en los ordenadores del laboratorio. El threshold calculado ha sido 310, y esta es la gráfica asociada:



Las observaciones son similares a las del threshold previo, con las irregularidades esperables en un entorno práctico real.

Ejercicio 3

Para el cálculo de los tiempos hemos optado por una matriz 2048x2048 y una matriz 1024x1024.

1024x1024

Tiempo de ejecución (s)

Versión \ # hilos	1	2	3	4
Serie	7.45	-	-	-
Paralela - bucle 1	7.43	4.23	3.25	2.78
Paralela - bucle 2	7.2	3.83	2.72	2.12
Paralela - bucle 3	7.11	3.61	2.52	1.82

Speedup

Versión \ # hilos	1	2	3	4
Serie	1,00	1,00	1,00	1,00
Paralela - bucle 1	0.96	1.68	2.17	2.53
Paralela - bucle 2	1.01	1.31	1.91	2.55
Paralela - bucle 3	1.07	1.95	2.78	3.79

2048X2048

Tiempo de ejecución (s)

Versión \ # hilos	1	2	3	4
Serie	64,81	-	-	-
Paralela - bucle 1	46.92	25.3	18.47	15.53
Paralela - bucle 2	45.65	23.57	16.74	13.38
Paralela - bucle 3	45.46	27.53	18.9	14.49

Speedup

Versión \ # hilos	1	2	3	4
Serie	1,00	1,00	1,00	1,00
Paralela - bucle 1	1.43	2.64	3.61	4.30
Paralela - bucle 2	1.48	2.43	3.51	4.6
Paralela - bucle 3	1.47	2.84	3.99	4.99

3.1 ¿Cuál de las tres versiones obtiene peor rendimiento? ¿A qué se debe? ¿Cuál de las tres versiones obtiene mejor rendimiento? ¿A qué se debe?

La versión que peor rendimiento tiene es la versión de serie. Se debe a que al no paralelizar nada estamos desperdiciando mucha capacidad de procesamiento. La versión que más rendimiento obtiene es la que paraleliza el bucle más externo ya que no tenemos que crear, sincronizar y destruir tantos hilos como si paralelizamos el bucle intermedio o el bucle interno.

Para paralelizar el bucle for externo solo tenemos que crear, sincronizar, destruir... 4 hilos que son los que tiene el procesador del ordenador en el que estamos realizando las pruebas.

Para el bucle intermedia tenemos que crear, sincronizar, destruir... $4 \cdot \text{num_iteraciones}$ hilos. Y para el bucle interno $4 \cdot \text{num_iteraciones}^2$.

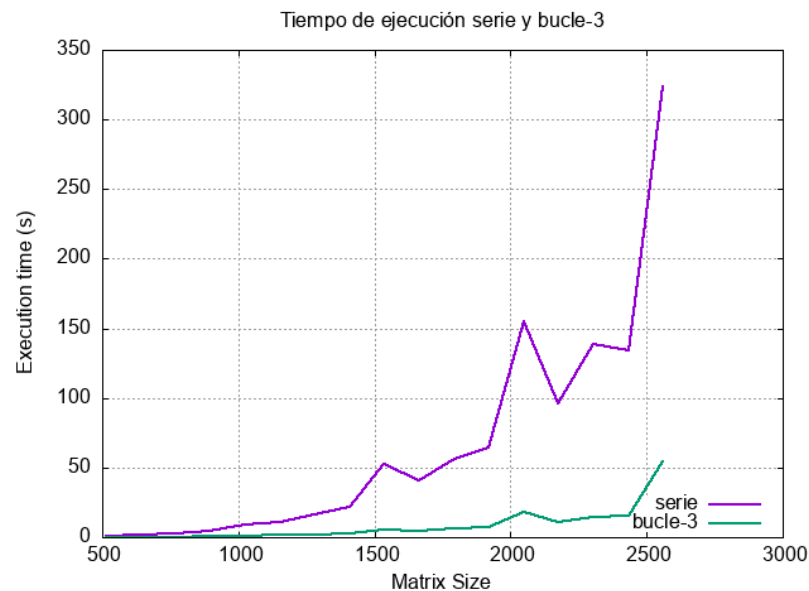
Para ponerlo en perspectiva si tenemos matrices 1000x1000 en el bucle interno tenemos que gestionar 4 hilos 1.000.000 de veces (4 millones de hilos).

3.2 En base a los resultados, ¿cree que es preferible la paralelización de grano fino (bucle más interno) o de grano grueso (bucle más externo) en otros algoritmos?

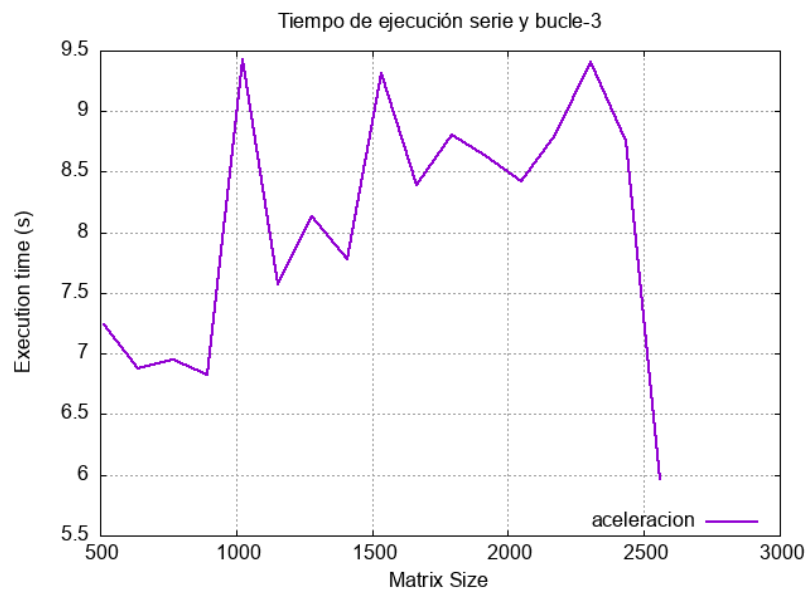
En base a los resultados es preferible la paralelización de grano grueso aunque esto puede depender del algoritmo ya que en algoritmos más complejos en los que haya dependencias entre iteraciones puede ser más difícil de implementar la paralelización de grano grueso.

3.3 Realice una gráfica de la evolución del tiempo de ejecución y la aceleración

Dado que la gráfica generada con los parámetros del enunciado muestra un comportamiento muy inestable en el tiempo de ejecución aumentamos el tamaño máximo de las matrices a 2560 y reducimos el tamaño del paso a 128. Con estos parámetros las gráficas resultantes ilustran mejor los resultados obtenidos.



Podemos ver como el tiempo de ejecución asciende cada vez más rápido pero con muchas irregularidades y que la diferencia entre el tiempo de ejecución entre la versión de serie y la versión con el bucle externo paralelizado aumenta cada vez más.



Vemos como la aceleración parece tener una tendencia ascendente pero en la última iteración la aceleración desciende abruptamente.

No fuimos capaces de estabilizar la aceleración a pesar de incrementar el valor de N y reducir el tamaño del paso.

EJERCICIO 4

Pregunta 4.1: ¿Cuántos rectángulos se utilizan en la versión del programa que se da para realizar la integración numérica? ¿Cuál es entonces el valor de h (ancho del rectángulo)?

Se utilizan $n = 10^8$ rectángulos, y puesto que integramos de 0 a 1, el ancho de un rectángulo será 10^{-8} .

Pregunta 4.2: Ejecute todas las versiones paralelas y la versión serie. Analice el rendimiento (en términos de tiempo medio de ejecución y la aceleración o speedup) y si el resultado obtenido es correcto o no, reflejando estos datos en una tabla. En las versiones que el resultado no sea correcto, incluya una breve explicación de por qué no es correcto.

Hemos creado un script de bash, "tiempos_versiones.sh" que ejecuta cada versión 10 veces, y calcula la media y el speedup.

Versión	Tiempo medio de ejecución (s)	Speedup (resp. versión en serie)	Es correcto
Serie	0,237	1	☒
1	0,216	1,097	☒
2	0,236	1,004	☒
3	0,212	1,117	☒
4	0,151	1,569	☒
5	0,148	1,600	☒
6	0,231	1,025	☒
7	0,155	1,532	☒

Ejercicio 4.3: En la versión *pi_par2*, ¿tiene sentido declarar *sum* como una variable privada? ¿Qué sucede cuando se declara una variable de tipo puntero como privada?

Si *sum* fuese privada, entonces el valor de *sum* en cada hilo no mantendría el valor original del puntero. Esto haría, en la mayoría de los casos, que *sum* apuntase a direcciones de memoria diferentes, lo que generaría comportamientos inesperados en el programa.

Ejercicio 4.4: ¿Cuál es la diferencia entre las versiones *pi_par5*, *pi_par3* y la versión *pi_par1*? Explique lo que es False sharing e indique qué versiones se ven afectadas por ello (y en qué medida). ¿Por qué en *pi_par3* se obtiene el tamaño de línea de caché?

El false sharing consiste en que variables distintas ocupan posiciones consecutivas en la caché. Si cada hilo accede a una de estas variables, uno podría pensar que no ocurrirían problemas, ya que cada hilo accede a la suya y no a las otras. El problema es que si las variables están en la misma

línea de caché, se forma un cuello de botella porque varios hilos tratan de acceder a la misma línea.

La versión *pi_par1* sufre este false sharing, pues cada hilo guarda sus resultados en posiciones consecutivas del array sum, que muy probablemente se encuentran en la misma línea de caché, lastrando el rendimiento.

pi_par3, por su parte, obtiene el tamaño de línea y lo divide entre el tamaño del dato (double). Así, calcula cuantos datos caben por línea de caché, y añade padding al array sum para que cada hilo escriba en una línea de caché distinta, previniendo el false sharing y mejorando el rendimiento.

Por último, *pi_par5* evita este conflicto empleando una alternativa al array. Básicamente, cada hilo crea su propia variable privada, acumula en ella la suma parcial, y luego se la suma al total, 'pi', que es una variable compartida. Gracias a la directriz critical, nos aseguramos de que no se van a dar condiciones de carrera ni problemas con la escritura simultánea. Sí que podría ocurrir que varios hilos intentasen escribir a la vez y tuviesen que esperar un poco, pero esta espera no supone una fracción del tiempo total significativa.

Ejercicio 4.5: Explique el efecto de utilizar la directiva critical. ¿Qué diferencias de rendimiento se aprecian? ¿A qué se debe este efecto?

La directiva critical permite combinar las sumas parciales de forma segura. Como hemos comentado, esto puede generar un pequeño cuello de botella, además de introducir un ligero overhead para la sincronización, pero estos efectos son mínimos y no lastran el rendimiento. Al contrario, *pi_par5* es más rápido que *pi_par3*. Esto podría deberse a que *pi_par3* accede constantemente al array, y además para calcular el índice realiza una multiplicación. *pi_par5*, en cambio, suma todo en una variable local, y sólo al final combina las sumas parciales.

Ejercicio 4.6: Ejecute la versión *pi_par6* del programa. ¿Qué diferencias de rendimiento se aprecian? ¿A qué se debe este efecto?

pi_par6 es más lenta que las otras versiones probadas. Esto se debe ,por un lado, a que vuelve a padecer false sharing, ya que cada hilo escribe en una posición del array sum y estas son consecutivas. Por otro lado, realiza un uso ineficiente de la directiva *omp_for*, ya que fuerza la sincronización de los hilos.

Ejercicio 4.7: ¿Qué versión es la óptima y por qué?

En nuestras pruebas, la versión que más rápida ha resultado es la 5. Ya la hemos comparado con la 1 y la 3, y hemos discutido por qué la 6 es más lenta. La prueba 2 es muy similar a la 1, y de hecho se observa un rendimiento muy cercano. La versión 4 sigue una lógica muy parecida a la del 5, acumulando las sumas parciales en una variable privada para reducir el false sharing. Por último, la versión 7 utiliza *reduction*, que es una instrucción nativa de *omp*, logrando un rendimiento muy cercano a la versión 5, ya que *omp* se encarga de optimizarla.

EJERCICIO 5

En el fichero `edgeDetector.c` se incluye un programa que aplica un algoritmo de procesamiento de imágenes para detectar bordes.

0. Compile y ejecute el programa usando como argumento una imagen de su elección. Examine los ficheros que se hayan generado y analice el programa entregado

Al compilar y ejecutar el programa vemos que se generan tres archivos cuyo nombre empieza por el nombre de la imagen pasada como argumento y con las extensiones `_grad.jpg`, `_grey.jpg` y `_grad_denoised.jpg`.

El archivo generado con extensión `_grey.jpg` aplica un filtro de grises que recuerda al ya estudiado en la práctica 2.

El archivo generado con extensión `_grad.jpg` pone toda la imagen de color negro salvo por los bordes, los cuales no son muy nítidos y parecen contener cierto error detectando como borde algo que no lo es. Esto se puede deber a que el algoritmo detecta en ese píxel un cambio en la tonalidad de la imagen y por eso lo detecta como borde.

En cambio, la imagen con extensión `_grad_denoised.jpg` aplica un filtro que reduce el “ruido” que se contemplaba antes y aumenta el contraste de los bordes con el fondo negro, ensanchando los bordes en el proceso.

1. El programa incluye un bucle más externo que itera sobre los argumentos aplicando los algoritmos a cada uno de los argumentos (señalado como Bucle 0). ¿Es este bucle el óptimo para ser paralelizado?

Responda a las siguientes cuestiones para complementar su respuesta.

a. ¿Qué sucede si se pasan menos argumentos que número de cores?

Si paralelizamos el bucle 0 y pasamos menos argumentos que número de cores de nuestro procesador no estaríamos aprovechando toda la capacidad de procesamiento disponible, haciendo que el programa sea poco eficiente.

b. Suponga que va a procesar imágenes de un telescopio espacial que ocupan hasta 6GB cada una, ¿es la opción adecuada? Comente cuanta memoria consume cada hilo en función del tamaño en pixeles de la imagen de entrada

En un ordenador convencional no seríamos capaces de llevar a cabo la ejecución ya que no podríamos mantener todas las imágenes procesadas en memoria. Si tenemos una imagen de dimensiones $n \times m$, la memoria utilizada por cada hilo es de $n \cdot m$ bytes. Luego no es la opción adecuada.

2. Durante la práctica anterior, observamos que el orden de acceso a los datos es importante. ¿Hay algún bucle que esté accediendo en un orden subóptimo a los datos? Corríjalo en tal caso.

a. Es imprescindible que el programa siga realizando el mismo algoritmo, por lo que solo se deberían realizar cambios en el programa que no cambien la salida.

b. Explique por qué el orden no es el correcto en caso de cambiarlo.

Bucle de RGB a escala de grises: Se comete el mismo error que en la práctica 2, y ahora sabemos que también afecta a la caché ya que por cada iteración del bucle interno no accedemos a la posición adyacente sino a una posición alejada lo que va a provocar fallos de escritura en la caché.

Sobel edge detection: De nuevo en este bucle si cambiamos el acceso en vez de por filas por columnas el algoritmo es ligeramente más rápido, teóricamente los accesos se realizan entre variables ubicadas más cerca. También pensábamos que crear variables locales que almacenen los datos de los pixeles sería más eficiente pero tras probar nos dimos cuenta de que no es así.

Median denoising filter: se accede a índices de la forma $p1 + p2 \cdot \text{width}$, siendo $p2$ la variable que se incrementa en el bucle interno. Esto se traduce en que vamos saltando de width en width , lo que genera fallos en caché que podemos evitar invirtiendo los bucles, para así avanzar en el índice de uno en uno. Cambiando el bucle externo por el interno, comprobamos que el rendimiento mejora ligeramente.

Gaussian denoising filter: Hemos cambiado el orden de acceso en los bucles internos para iterar externamente sobre $p2$ e internamente sobre $p1$ reduciendo los fallos de cache en el acceso a $\text{kernel}[p1 + p2 * (2 * \text{radius} + 1)]$. En los bucles externos recorremos por filas para reducir los fallos de cache en $\text{edges_denoised}[x + y * \text{width_denoised}]$.

3. Obviando el Bucle 0, pruebe diferentes paralelizaciones con OpenMP comentando cuales deberían obtener mejor rendimiento.

En el ejercicio 3 hemos aprendido que cuando nos encontramos con bucles for anidados, es mejor en términos de rendimiento paralelizar el bucle más externo. Vamos a aplicar esta idea a los bucles con alta carga de procesamiento de nuestro programa.

Bucle de RGB a escala de grises: El bucle externo es fácil de paralelizar y aumenta ligeramente el rendimiento del programa por lo que sale rentable paralelizarlo.

Sobel edge detection: De nuevo el bucle externo es óptimo para la paralelización ya que consume gran parte del procesamiento al realizar cálculos complejos. Al ejecutar el programa con el bucle paralelizado notamos una mejoría en el tiempo de ejecución.

Classic gaussian filter: Para paralelizar el bucle externo simplemente tuvimos que privatizar algunas variables que se usaban en los bucles internos. La paralelización de este bucle es la que mayor mejoría aporta al programa.

Median denoising filter: En este caso no es posible paralelizar ni el bucle más externo ni el siguiente a este debido a que se realiza un qsort después del bucle interno sobre el array lo que provoca que los hilos escriban mientras el array se está ordenando resultando en error. Por otro lado, si paralelizamos el bucle siguiente el coste de gestión de los hilos es superior al rendimiento obtenido.

(Respuesta conjunta de 4 y 5)

4. Rellene una tabla con resultados de tiempos y speedup respecto a la versión serie para imágenes de distintas resoluciones (SD, HD, FHD, UHD-4k, UHD-8k). Añada a su vez una columna que sea la tasa de fps a la que procesaría el programa.

5. Algo que hemos dejado de lado es utilizar las optimizaciones del compilador. Repita el apartado anterior añadiendo al comando gcc la bandera -O3. Investigue sobre las optimizaciones que implementa el compilador (y como podrían afectar a nuestra paralelización). ¿Implementa el desenrollado de bucles? ¿Está activada esta opción? ¿Implementa algún tipo de vectorización? Nota: la opción -O3 puede generar warnings al ser compilado. Compruebe que la salida sigue siendo la misma en todo caso.

Tiempo y FPS								
Resolución	Median Serie O0	Gaussian Serie O0	Median Omp O0	Gaussian Omp O0	Median Serie O3	Gaussian Serie O3	Median Omp O3	Gaussian Omp O3
SD	0.189683 5.271	0.032372 30.890	0.185741 5.383	0.005970 167.504	0.153285 6.523	0.007452 134.192	0.152881 6.541	0.001621 616.903
HD	0.700 1.427	0.130 7.649	0.678 1.473	0.023109 43.273	0.574 1.740	0.034 28.850	0.561 1.780	0.006 166.002
FHD	1.531 0.653	0.302 3.305	1.470 0.680	0.051 19.305	1.263 0.791	0.087 11.391	1.226 0.815	0.013 75.580
4k	5.859	1.335	5.550	0.207	4.964	0.434	4.648	0.052

	0.170	0.748	0.180	4.817	0.201	2.300	0.215	19.142
8k	22.636	5.583	21.479	0.829	18.575	1.869	17.957	0.208
	0.044	0.179	0.0465	1.204	0.053	0.535	0.055	4.800

El speedup lo hemos calculado respecto a la versión en serie, sin optimizaciones (O0), y utilizando median, ya que es la más lenta, por lo que tiene sentido ver cómo de rápida son las otras versiones respecto a la más lenta:

Speedup resp. median serie O0								
Resolución	Gaussian Omp O0	Median Omp O0	Gaussian Serie O0	Median Serie O0	Gaussian Omp O3	Median Omp O3	Gaussian Serie O3	Median Serie O3
SD	31.772	1.021	5.859	1	117.016	1.240	25.453	1.237
HD	30.322	1.032	5.359	1	116.322	1.247	20.216	1.219
FHD	29.563	1.041	5.061	1	115.742	1.248	17.445	1.211
4k	28.229	1.055	4.386	1	112.167	1.260	13.483	1.180
8k	27.276	1.053	4.054	1	108.665	1.260	12.111	1.218

Atendiendo al informe del compilador, observamos que efectivamente la versión -O3 implementa vectorización y desenrollamiento de bucles.